KEPT PORTRAITS · KEPTPORTRAITS.COM

Scalaro Mailer — Deployment Record

Date: 26–29 September 2026

CONTENTS

1. What is running
   - Why the API and the worker are separate processes
2. The public hostname
3. Sending: Resend, not SES
   - The one thing that differs
   - Not yet built
4. Problems found and fixed
   - Compose read the wrong .env
   - The worker reached for a registry with no image
   - Certificates: the renewal that could not work
   - Nothing reloaded nginx after a renewal
   - A redundant cron failing twice a day
   - scalaro.io was failing on a dead subdomain
5. Known-broken, left alone deliberately
   - The trap to remember
6. Routine operations
   - nginx
7. Secrets
8. What to do next
9. Deploying anything else on this server

Server: Hetzner VPS, 209.145.55.76 **Repository:** github.com/yaseenishfaqjan/email-marketing-tool

This is what was deployed, how it works, and what to do the next time — whether that is onboarding a second brand, deploying another application on this server, or picking the work back up after a gap.

1. What is running

Three containers, built from this repository, in `/opt/mailer`:

CONTAINER	ROLE
`mailer-api`	HTTP API, signup forms, tracking links, unsubscribe, dashboard
`mailer-worker`	Drains the send queue, runs automations, enforces warm-up caps
`mailer-db`	PostgreSQL 16, five migrations applied

No host ports are published. `mailer-api` joins `scalaro-net`, the Docker network the shared nginx container already sits on, so nginx reaches it as `http://mailer-api:8080`. On a box already running fifteen applications this removes the question of which port is free, and nothing is exposed to the internet except through nginx.

The database is deliberately **not** on `scalaro-net`. Nothing outside this stack needs to reach it.

Why the API and the worker are separate processes

The API answers in milliseconds; the worker spends minutes draining a broadcast. One process would make the signup forms slow during a send, and restarting to deploy an API fix would interrupt one.

---

2. The public hostname

`links.scalaro.io` → 209.145.55.76, TLS via Let's Encrypt, valid to 26 December 2026.

Only these paths reach the application:

```
/o/        open pixel
/c/        click redirect
/u/        one-click unsubscribe
/confirm/  double opt-in confirmation
/f/        signup forms and their submit endpoint
/webhooks/ses
/health
```

Everything else returns 404.

This hostname appears in every email that goes out — printed in footers, hovered over by recipients, crawled by spam filters. The admin API and the dashboard are deliberately **not** reachable on it. Reach them from inside the Docker network instead:

```
docker run --rm --network scalaro-net curlimages/curl:latest -s \
  http://mailer-api:8080/health
```

---

3. Sending: Resend, not SES

The platform was built for Amazon SES. SES declined production access on 28 September, with no stated reason — the usual read for a days-old account: no billing history, a first request typed as MARKETING rather than transactional, and a description of five brands consolidating that reads, to a reviewer, like bulk mail.

Only one file ever talked to a provider, so the fix was an adapter rather than a rewrite:

```
                  src/sending/provider.mjs
                     |            |
            src/sending/ses.mjs    src/sending/resend.mjs
                    |                    |
              Amazon SES              Resend
```

`EMAIL_PROVIDER=resend` in `/opt/mailer/.env`, and a restart. Switching back when SES is approved is the same one word. Everything upstream of the adapter — rendering, merge fields, suppression, warm-up caps, automations, signup forms, unsubscribe tokens — is identical either way.

The worker's startup line names the active provider, so the logs confirm it:

```
[worker] started — resend at 8/s in batches of 100, automations every pass
```

The one thing that differs

Resend has no raw-MIME endpoint. Under SES we write `List-Unsubscribe` and `List-Unsubscribe-Post` ourselves; under Resend we pass them as custom headers and Resend writes them. Same result on the wire, and `test/resend.test.mjs` asserts both reach the request body — Gmail and Yahoo have required one-click unsubscribe from bulk senders since February 2024, and losing it fails quietly.

Not yet built

Resend webhooks. `/webhooks/ses` handles SES events, so hard bounces and complaints suppress automatically under SES. Nothing consumes Resend's webhook format yet.

Sending through Resend today therefore means **bounces and complaints are not automatically suppressed**. At first-brand volumes that is survivable if somebody watches the Resend dashboard. It is not survivable at scale, and it should be built before a second brand is onboarded.

---

4. Problems found and fixed

Six real defects surfaced during deployment. They are recorded because each one would otherwise have been found later, under worse conditions.

Compose read the wrong `.env`

`docker compose` looks for a `.env` beside the compose file for `${VAR}` interpolation. Ours lives at `/opt/mailer/.env`, one level up, so `${POSTGRES_PASSWORD}` resolved to empty and `DATABASE_URL` came out malformed. `env_file:` only fills the container's environment; it has no effect on interpolation.

Every compose command now carries `--env-file ../.env`:

```
cd /opt/mailer/deploy
docker compose --env-file ../.env -f docker-compose.prod.yml up -d --build
```

The worker reached for a registry with no image

`mailer-api` builds `mailer:latest`; `mailer-worker` declared the same image with no build section, so compose tried to pull it and printed `pull access denied for mailer` on every deploy before falling through to the local build. `pull_policy: never` says out loud that the image is built here.

Certificates: the renewal that could not work

The playbook claimed every certificate on this box used `webroot`. The real split was 16 standalone, 3 webroot — and two global hooks stop and start nginx around every renewal so the standalone ones can bind port 80.

That makes **webroot the broken choice here**, not the safe one: the pre-hook stops the only thing that could serve the ACME challenge, and validation fails with `Connection refused`. The certificate issued for `links.scalaro.io` with `--webroot` would have failed to renew in December. A dry run caught it.

All 19 certificates are now standalone. Issue new ones the same way:

```
certbot certonly --standalone -d yourdomain.com \
  --email yasinishfaq5@gmail.com --agree-tos --non-interactive
```

The pre-hook stops and starts nginx, so nothing needs stopping by hand. The cost is ~30 seconds of downtime across all fifteen applications per renewal.

Nothing reloaded nginx after a renewal

nginx reads certificates once, at startup. A renewed certificate would have sat on disk unused until something restarted the container — which on this box could be months. Fixed with a deploy hook at `/etc/letsencrypt/renewal-hooks/deploy/reload-nginx.sh`, guarded so it does nothing while nginx is stopped (deploy hooks run *before* the post-hook).

A redundant cron failing twice a day

A root crontab entry ran `certbot renew --standalone --quiet` at 02:30 and 14:30. `certbot.timer` already runs `certbot renew`, which uses each certificate's own recorded authenticator. Removed. A job that fails sixty times a month trains you to ignore certbot errors, which is how the real one gets missed.

`scalaro.io` was failing on a dead subdomain

The certificate covered `scalaro.io`, `www.scalaro.io` and `api.scalaro.io`. `api.scalaro.io` has no DNS record, and Let's Encrypt validates **every** name on a certificate — one failure fails the whole thing. Reissued without it; valid to 26 December.

---

5. Known-broken, left alone deliberately

Six certificates still fail to renew. None serves live traffic, so when they expire on 1 December nobody notices:

CERTIFICATE	WHY
`api.fabrioza.com`	No DNS record
`apollo.nodevant.com`	No DNS record
`app.fairway360.io`	No DNS record
`test.amaanacademy.com`	No DNS record
`apolloixofficial.com`	Nameservers returning SERVFAIL
`globalshield360.io`, `nodevant.com`	Resolve to 95.216.22.216 (cPanel), served from there with cPanel's own certificates

If you ever point one of those domains at this VPS, reissue its certificate first — the one here will be expired by then.

The trap to remember

nginx blocks reference certificate paths. Delete a certificate while its block still exists and the **next nginx reload fails**, taking down all fifteen sites at whatever random moment someone next reloads.

The order is always: remove the nginx block → `nginx -t` → reload → *then* delete the certificate. Never the reverse.

---

6. Routine operations

All commands assume `cd /opt/mailer/deploy` first.

```
# Deploy a change
cd /opt/mailer && git pull
cd deploy && docker compose --env-file ../.env -f docker-compose.prod.yml up -d --build

# Logs
docker compose --env-file ../.env -f docker-compose.prod.yml logs -f mailer-worker

# Status
docker compose --env-file ../.env -f docker-compose.prod.yml ps

# Restart after an .env change
docker compose --env-file ../.env -f docker-compose.prod.yml up -d \
  --force-recreate mailer-api mailer-worker

# Reach the admin API
export ADMIN_TOKEN=$(grep '^ADMIN_TOKEN=' /opt/mailer/.env | cut -d= -f2-)
docker run --rm --network scalaro-net curlimages/curl:latest -s \
  http://mailer-api:8080/health
```

nginx

```
cd /opt/scalaro
docker compose exec nginx nginx -t        # always before a reload
docker compose exec nginx nginx -s reload
```

Config lives at `/opt/scalaro/nginx.conf`, mounted into the container as `/etc/nginx/conf.d/default.conf`. Back it up before editing:

```
cp /opt/scalaro/nginx.conf /opt/scalaro/nginx.conf.bak.$(date +%F)
```

---

7. Secrets

`/opt/mailer/.env`, mode `600`, never committed.

VARIABLE	NOTE
`TOKEN_SECRET`	**Permanent.** Signs every unsubscribe link. Rotate it and every link already sitting in an inbox stops working — which means unsubscribes start failing, which means spam complaints. Back it up somewhere you will still have in three years.
`ADMIN_TOKEN`	Bearer credential for the admin API
`POSTGRES_PASSWORD`	Appears twice: alone, and inside `DATABASE_URL`. They must match exactly.
`RESEND_API_KEY`	Sending-access key, scoped to `scalaro.io`
`AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY`	`mailer-ses` IAM user, `ses:SendEmail` only

Generate secrets as hex, not base64. Base64 emits `/` and `+`, and a `/` inside a Postgres connection URL breaks the parser:

```
openssl rand -hex 24
```

---

8. What to do next

In order.

1. **Create the first brand and send a test.** Nothing has been sent through this platform yet. Until one real email arrives in an inbox, the pipeline is unproven end to end.
2. **Build the Resend webhook handler.** `/webhooks/resend`, mirroring `/webhooks/ses`: verify the signature, write hard bounces and complaints to the global suppression list. Do this before the second brand.
3. **Onboard brands one at a time**, dry-running each contact list first. The importer reports malformed addresses, disposable domains, role accounts, duplicates and likely typos before anything is written.
4. **Re-apply to SES.** The adapter is already written and SES is roughly half the cost at 500k/month. What to change from the declined request: - Wait for a month or two of billing history - Request **transactional** first, not marketing - One brand, modest volume — not 500k/month across five - Make sure `scalaro.io` has a visible privacy policy, terms and contact page - State the verified identity exactly as the SES console shows it; the declined request said `mail.scalaro.io`, which may not be what is verified - Open a **new** request rather than replying to the closed case
5. **Deploy n8n properly.** A `scalaro-n8n` container is already running on the box. Before building workflows on it, pin `N8N_ENCRYPTION_KEY` and `WEBHOOK_URL` — without a pinned encryption key, saved credentials become unreadable on the next restart.

9. Deploying anything else on this server

The pattern this deployment followed, which is the one to copy:

1. Own directory under `/opt/<project>`, own compose file
2. Join `scalaro-net`, publish **no host ports** — nginx reaches the container by name
3. `.env` beside the compose directory, `chmod 600`, and every compose command carries `--env-file ../.env`
4. Named volumes for data, never bind mounts into the deploy directory — a redeploy replaces that directory
5. Certificate **first**, standalone, then the nginx block — nginx will not start with an `ssl_certificate` path that does not exist
6. `nginx -t` before every reload, always

Before picking a port for anything that does need one, check what is already taken:

```
docker ps --format '{{.Names}}\t{{.Ports}}' | sort
```

This deployment originally used 8090, which `southernroots-web-1` already held. That is how the no-host-ports approach was arrived at.